IY3660 – Applications of Cryptography

Week 4 Lecture
Block ciphers: modes of operation and padding

Elizabeth Quaglia
Information Security Group

ROYAL HOLLOWAY UNIVERSITY OF LONDON

In today's lecture

- Motivation for modes of operation
- ECB mode, CBC mode, CTR mode
- Attack on CBC mode with non-random IVs
- Padding and padding oracle attacks

Motivation for modes of operation

Motivation for modes of operation

A block cipher encrypts a message of exactly *n* bits

- What if the message length is not a multiple of *n* bits?
- What if the message length is greater (or smaller) that *n* bits?
- What if the message is not of a fixed length but actually a stream?

- Modes of operation provide different ways of using a block cipher to encrypt flexible amounts of data
  - Different performance characteristics
  - Different error-propagation properties
  - Different suitability for different applications

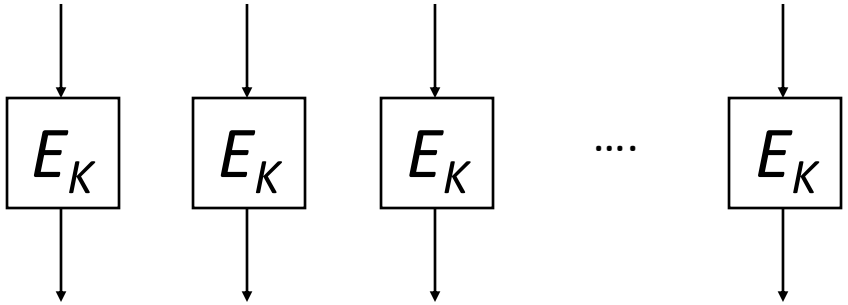
Main modes of operation

- NIST standard FIPS 81 (1980) specifies four modes for DES
  - ECB - Electronic Code Book
  - CBC - Cipher Block Chaining
  - CFB - Cipher Feedback
  - OFB - Output Feedback
- NIST SP 800-38A (2001): updated DES modes above and defined Counter Mode (CTR)
- ANSI X9.52 (1998) specifies seven modes for triple-DES
  - The first four modes above and three variants
- Most common modes now in use: CBC and CTR

ECB mode

Electronic Code Book (ECB)

- The simplest way to use a block cipher to encrypt longer messages

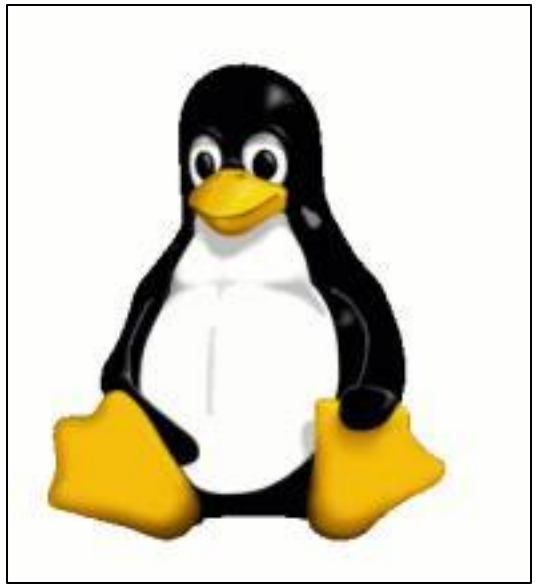


- Split message into blocks (of n bits, where n is the block cipher size)
  - If not multiple of n, **pad** the last block
- Encryption can be parallelised (leading to efficient implementations)
- Any error in a ciphertext block affects the decryption of a single block
  - i.e., limited error propagation

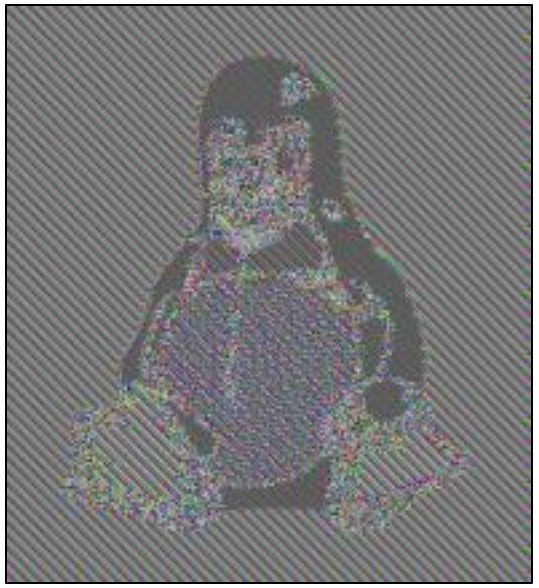
ECB information leakage

- For a fixed key *K*, a given block of plaintext is always encrypted in the same way to produce the same ciphertext block
- Encryption is **deterministic**
- Leads to serious information leakage in many applications
- ECB mode is very rarely the correct mode to use...

ECB information leakage – example



Tux the Penguin, the Linux mascot. Created in 1996 by Larry Ewing with The GIMP. lewing@isc.tamu.edu



ECB-Tux

Is anybody still using ECB today?

- Zoom (2020)

THE CITIZEN LAB munk school OF GLOBAL AFFAIRS & PUBLIC POLICY UNIVERSITY OF TORONTO RESEARCH NEWS ABOUT

Research < App Privacy and Controls

Move Fast and Roll Your Own Crypto

A Quick Look at the Confidentiality of Zoom Meetings

By Bill Marczak and John Scott-Railton April 3, 2020

Read our description of Zoom's waiting room vulnerability, as well as frequently asked question about Zoom and encryption issues.

This report examines the encryption that protects meetings in the popular Zoom teleconference app. We find that Zoom has "rolled their own" encryption scheme, which has significant weaknesses. In addition, we identify potential areas of concern in Zoom's infrastructure, including observing the transmission of meeting encryption keys through China.

Key Findings

- Zoom documentation claims that the app uses "AES-256" encryption for meetings where possible. However, we find that in each Zoom meeting, a single AES-128 key is used in ECB mode by all participants to encrypt and decrypt audio and video. The use of ECB mode is not recommended because patterns present in the plaintext are preserved during encryption.
- The AES-128 keys, which we verified are sufficient to decrypt Zoom packets intercepted in Internet traffic, appear to be generated by Zoom servers, and in some cases, are delivered to participants in a Zoom meeting through servers in China, even when all meeting participants, and the Zoom subscriber's company, are outside of China.

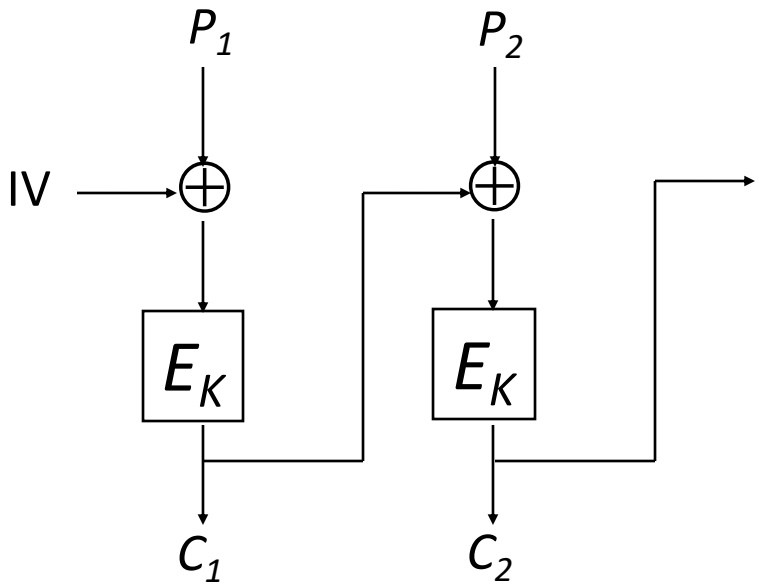
CBC mode

Cipher Block Chaining (CBC) mode

- Aims to hinder information leakage of ECB mode
- Uses previous ciphertext block (or IV if first block) to randomise the input to the block cipher at each application
- Not parallelisable

Encryption:

$$C_0 = IV$$
$$C_i = E_K(P_i \oplus C_{i-1})$$



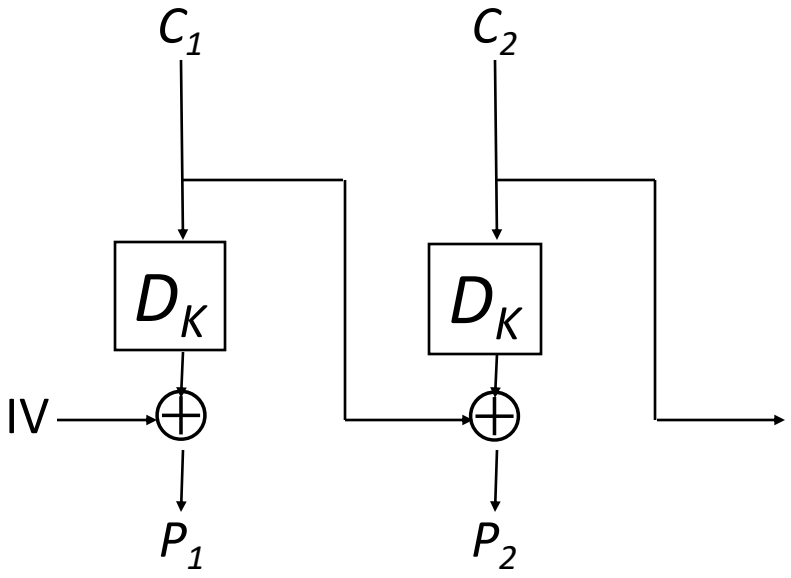
CBC mode decryption

Decryption:

$D_K(C_i) = P_i \oplus C_{i-1}$

and hence:

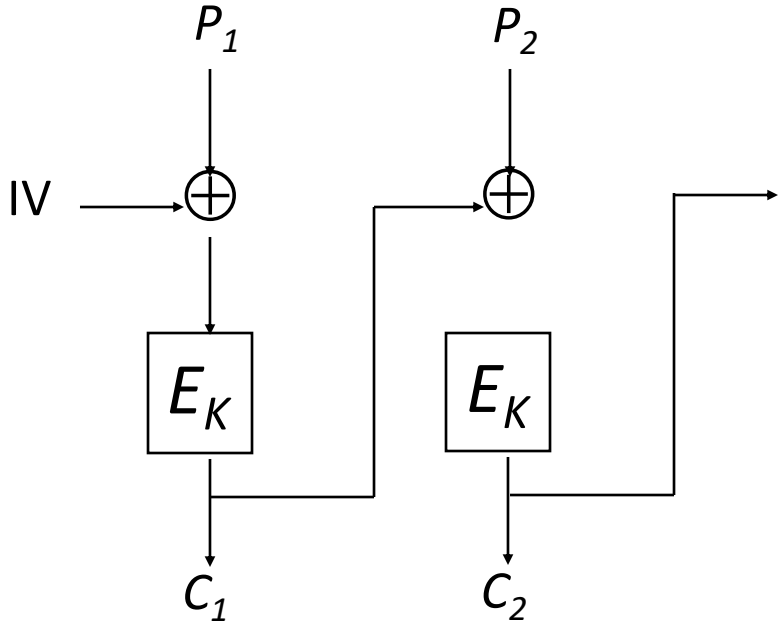
$P_i = D_K(C_i) \oplus C_{i-1}$



Encryption:

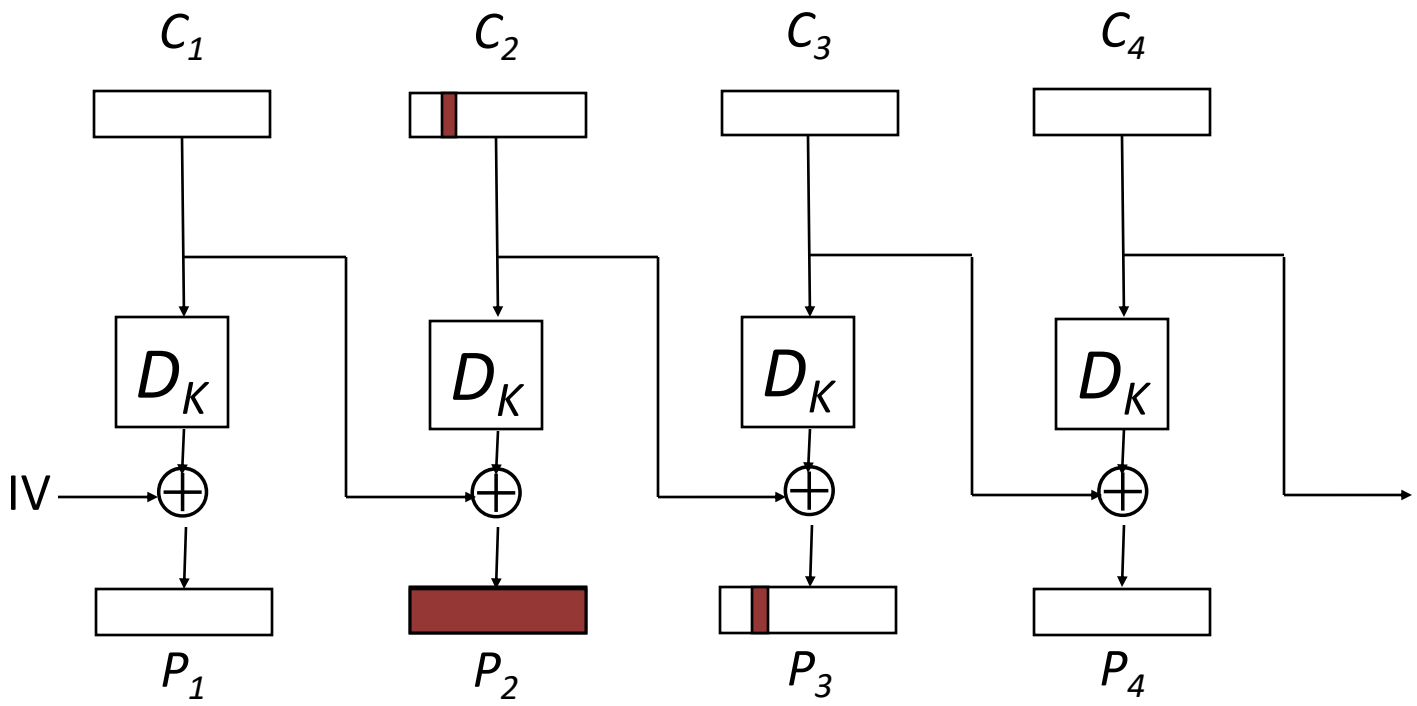
$C_0 = \text{IV}$

$C_i = E_K(P_i \oplus C_{i-1})$



Error propagation in CBC

- Suppose an error arises in ciphertext block C_2



- Then the error propagates to P_3, and P_2 gets randomised
- We will come back to this

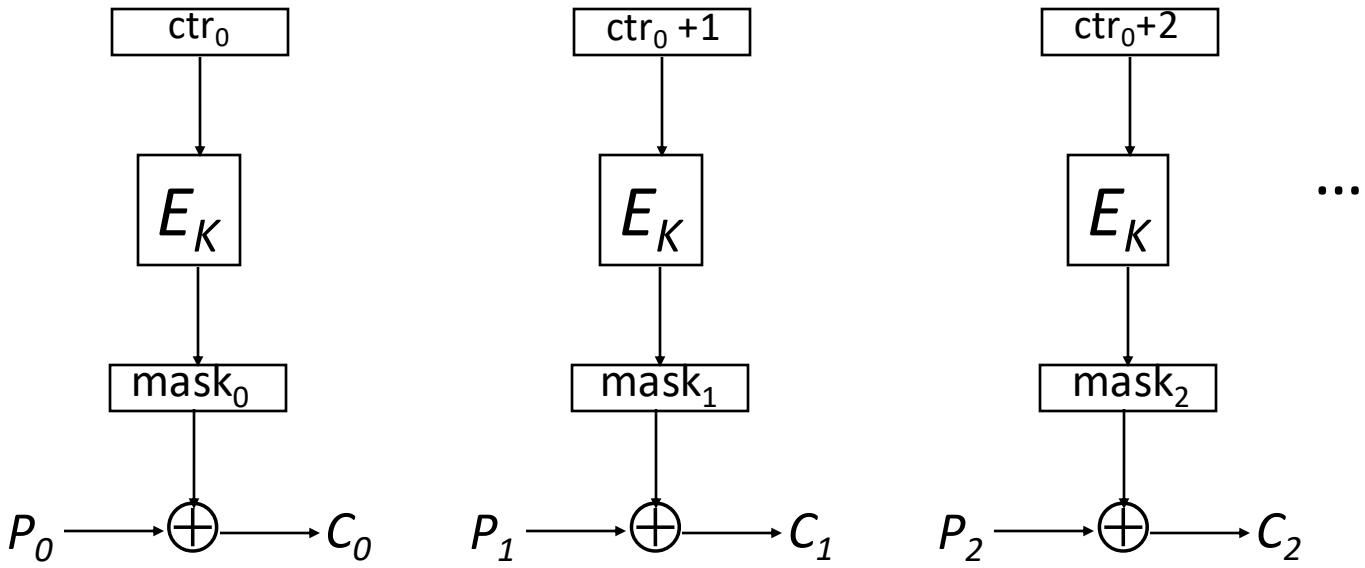
Further remarks on CBC mode

- CBC requires use of an Initialisation Vector (IV):
  - Needed for decryption so should be considered as part of the ciphertext: $C_0 = IV$
    - Implies ciphertext expansion: ciphertext will be at least one block larger than plaintext
  - IV needs to be ***random*** and ***unpredictable*** for each message
    - Implies requirement for good source of randomness
- ***Padding*** may be needed so plaintext is a whole number of blocks
- Both requirements are the source of practical attacks
- Note: CBC mode problematic if block size n is too small
  - Leads to block collisions: see https://sweet32.info/ for example

CTR mode

Counter (CTR) mode encryption

- ctr_i is set to be an incrementing counter: $\text{ctr}_i = \text{ctr}_0 + i$.
- No padding needed since last mask can be truncated to required length
- Only block cipher encryption operation needs to be implemented
- Parallelisable, can also pre-compute encryption masks before plaintext is known



Counter (CTR) mode encryption

- CTR is a stream cipher mode
  - It turns a block cipher into a stream cipher!
  - Block ciphers are usually slower than dedicated stream cipher designs, so there is still a place for stream ciphers in applications
    - Although on commodity Intel/AMD CPUs, where you have AES special instructions, it is hard to beat AES (and thus AES-CTR) for speed!
- Error propagation:
  - A bit-flip in the ciphertext leads to a bit-flip in the plaintext
  - More generally, XOR of a mask Δ with the ciphertext leads to the same mask Δ being XORed onto the plaintext

Counter (CTR) mode encryption

- Security requirement: for a fixed key K, the counter values used **must not** repeat!
  - As for keystream repeat issue for a stream cipher: XOR of ciphertexts then yields XOR of plaintexts
- Can achieve non-repeating property by using one of the following methods:
  - Starting with a random value for ctr_0 for each plaintext
    - Requires a good source of randomness, need to prevent colliding counters
  - Maintaining a record of the last value of ctr_i used
    - Requires state in encryption algorithm, state needs to survive across power cycles
  - Constructing ctr_i by concatenating a **per plaintext nonce** N supplied by the calling application and an internal counter (starting from zero for each new plaintext), so: $ctr_i = N \,||\, i$
    - requires nonce to not repeat, hence some kind of state needed in application

Attack on CBC mode with non-random IVs

Is anyone using CFB with a fixed IV?

- Microsoft (2020)

Zerologon:

Unauthenticated domain controller compromise
by subverting Netlogon cryptography (CVE-2020-1472)

Summary

This whitepaper describes some of the technical details of CVE-2020-1472 (which we have dubbed "Zerologon"), a critical vulnerability in Windows Server that has received a CVSS score of 10.0 from Microsoft. In order to mitigate this issue, it is highly recommended to install Microsoft's August 2020 security patches on all Active Directory domain controllers. Leaving a DC unpatched will allow attackers to compromise it and give themselves domain admin privileges. The only thing an attacker needs for that is the ability to set up TCP connections with a vulnerable DC; i.e. they need to have a foothold on the network, but don't require any domain credentials.

by Tom Tervoort, September 2020

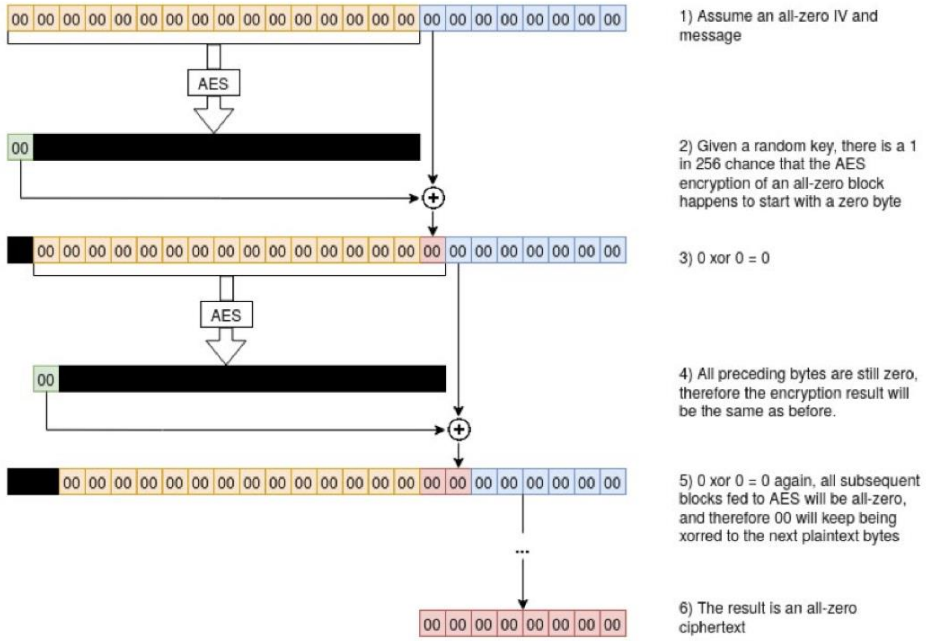
Secura

see: https://www.secura.com/pathtoimg.php?id=2055

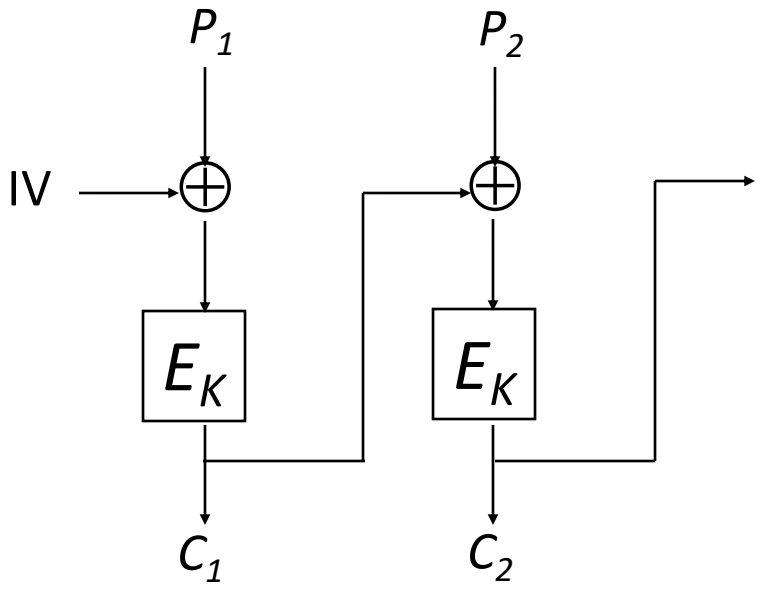
In order to be able to encrypt the initial bytes of a message, an Initialisation Vector (IV) must be specified to bootstrap the encryption process. This IV value must be unique and randomly generated for each separate plaintext that is encrypted with the same key. The *ComputeNetlogonCredential* function, however, defines that this **IV is fixed and should always consist of 16 zero bytes**. This violates the requirements for using AES-CFB8 securely: its security properties only hold when IVs are random.

So, is this actually a problem here? What can go wrong with an all-zero IV? Because of the obscurity of CFB8, I could not find any literature on this subject. So I tried to come up with some *chosen-plaintext attacks* myself and figured out something interesting: **for 1 in 256 keys, applying AES-CFB8 encryption to an all-zero plaintext will result in all-zero ciphertext**. Figure 3 shows why this is the case.

AES-CFB8 encryption (all-zero IV and plaintext)



Recall: Cipher Block Chaining (CBC) mode



Attack on CBC mode with non-random IVs

- Recall that in CBC mode the Initialisation Vector IV (the *block* C_0) needs to be **random/unpredictable**
  - This is required by theory to get security proofs for CBC mode to work
  - But is also important in practice
- We show a **chosen plaintext distinguishing attack** on CBC mode with predictable IVs
  - First observed by Rogaway in 1995
  - In some circumstances, this can be amplified to yield a **full plaintext recovery attack**!

http://web.cs.ucdavis.edu/~rogaway/papers/draft-rogaway-ipsec-comments-00.txt

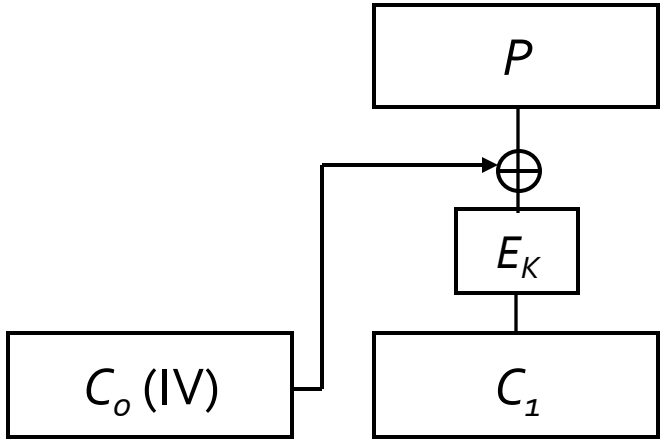
Attack on CBC mode with non-random IVs

- Suppose, for simplicity, that only single-block plaintexts *P* will be encrypted
- Suppose the adversary knows that either P_1 or $\overline{P_1}$ has been encrypted (e.g., "YES" or "NAY"), and wants to determine which one it was
- We assume the adversary can get plaintexts of its choice encrypted – so this is a **chosen plaintext** attack
  - Note: the adversary does not choose the IV - it is generated by the encryption oracle

Attack on CBC mode with non-random IVs

The attacker

- Receives ciphertext C (= blocks C_0, C_1)* for message P
- Knows P is equal to either P_1 or $\overline{P_1}$ and wishes to determine which one

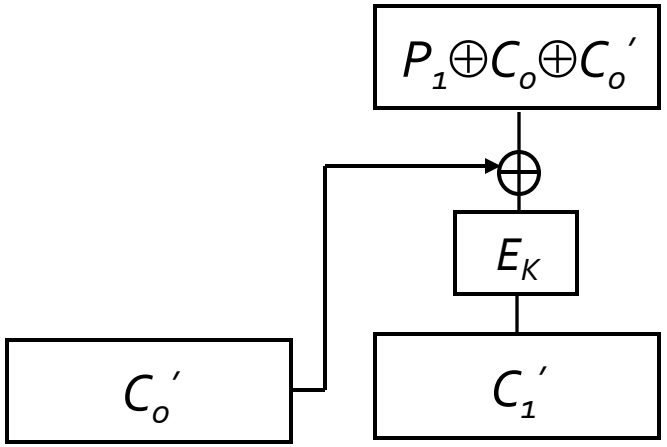


*recall, we treat the IV as part of the ciphertext in CBC (that is, $C_0 = IV$)

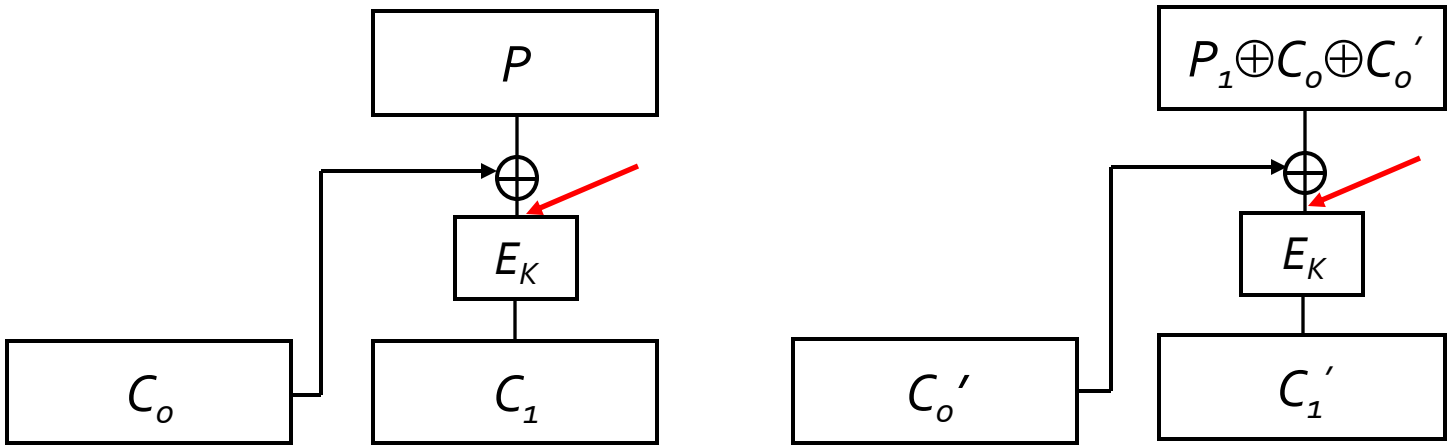
Attack on CBC mode with non-random IVs

1. Attacker predicts that C_0' will be used as the IV for the next encryption.
2. Attacker now requests encryption of the block $P' = P_1 \oplus C_0 \oplus C_0'$
3. Attacker receives (from the oracle) a ciphertext C_0', C_1'

In step 2 the attacker is guessing that the correct plaintext block is P_1 they will use the ciphertext from step 3 to verify their guess.



Attack on CBC mode with non-random IVs



- If $P = P_1$, then the inputs to the block cipher are the same at the red arrows
  - Hence, if $P = P_1$, then $C_1 = C_1'$
  - otherwise, if $P = \overline{P_1}$, the inputs are different, so we have $C_1 \neq C_1'$

- Comparing C_1 and C_1' means the adversary can test whether P_1 or $\overline{P_1}$ was encrypted!

Attack on CBC mode with non-random IVs

- Attack extends easily to multiple-block messages
- Attack applies in case of **IV chaining**, where IV for current message is set to be last ciphertext block from previous message
    - As used in SSL 3.0, TLS 1.0
    - See Dai and Moller, 2004
- Attack extends to a plaintext recovery attack when the plaintext has **low entropy**, rather than just 2 possibilities
- In some cases, the attack can be extended to **full plaintext recovery**
    - See the BEAST attack on SSL/TLS: Duong and Rizzo, 2011

Padding and padding oracle attacks

Padding in CBC mode

- So far, we assumed all plaintexts have bit-lengths that are multiples of the block size n
- In practice, we want to be able to encrypt arbitrary length messages
- Common solution: use message padding

Message padding:

- Applies an **injective** map pad( ) to messages prior to CBC mode encryption
- Remove padding after decryptions
- Properties of pad()
    - Takes inputs in $\{0,1\}^*$ (a bit-string of arbitrary bit-length)
    - Has outputs in $\{\{0,1\}^n\}^*$, that is, the set of all bit-strings that are multiples of n in length
    - Is necessarily expanding
    - Must be and its inverse must be efficiently computable
    - May be randomised or deterministic

Concrete example: simplified TLS padding

- The message is always a whole number of bytes (multiple of 8 bits)
- pad( ) adds $(t+1)$ copies of byte value t, where $0 \leq t < n/8$, to bring message length up to multiple of n bits ($n/8$ bytes)
- This creates a plaintext that can then be CBC mode encrypted
- For AES, where $n = 128$, we have $n/8 = 16$ and the possible pads are:

0x00

0x01 0x01

…

0x0f 0x0f … 0x0f (16 copies of 0x0f)

Concrete example: simplified TLS padding

- Note that at least one byte of padding is **always** added
  - Again: padding is *always* expanding
- If message length is already an exact multiple of n bits, then a completely new block containing only padding is added (0x0f … 0x0f block for $n = 128$)
- This is just an example; many other padding functions are possible
- Summary
  - <u>Enc:</u> add padding to message to generate plaintext; encrypt with CBC
  - <u>Dec:</u> perform CBC mode decryption, remove padding to recover the message

It looks fine; what can go wrong?

Padding oracle attacks

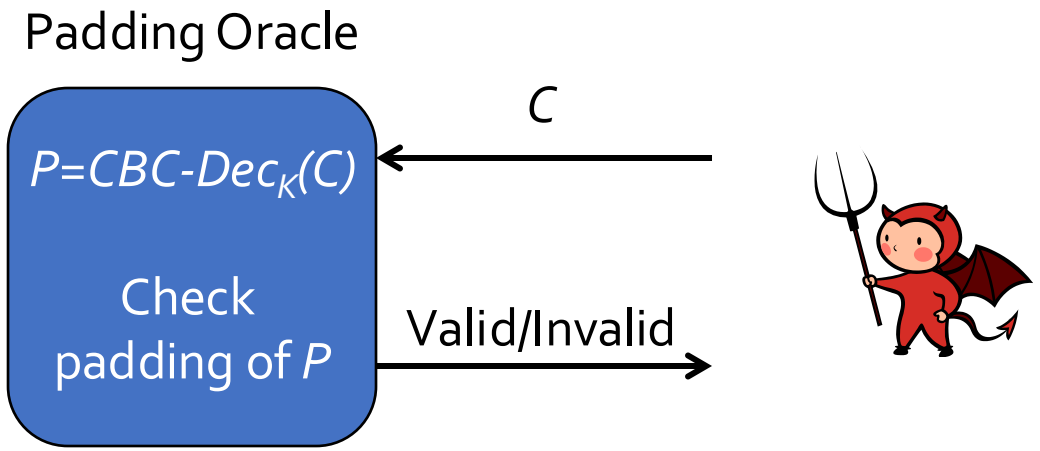
- After CBC mode decryption, message handling code needs to parse plaintext as: `message||padding`, then remove `padding`, and finally pass `message` to application
- With simplified TLS padding, `padding` should only be one of 16 possibilities:

  "0x00" or "0x01 0x01" or … or "0x0f 0x0f… 0x0f".

  - What to do if the padding field is not one of these?
- Typical implementation will throw an exception with a helpful error message if padding is not one of the expected values
  - Crypto code should not return garbage data to calling application
- An adversary may be able to detect when this happens, via
  - Error message visible on the network
  - System logs
  - Timing behaviour

Padding oracle attacks

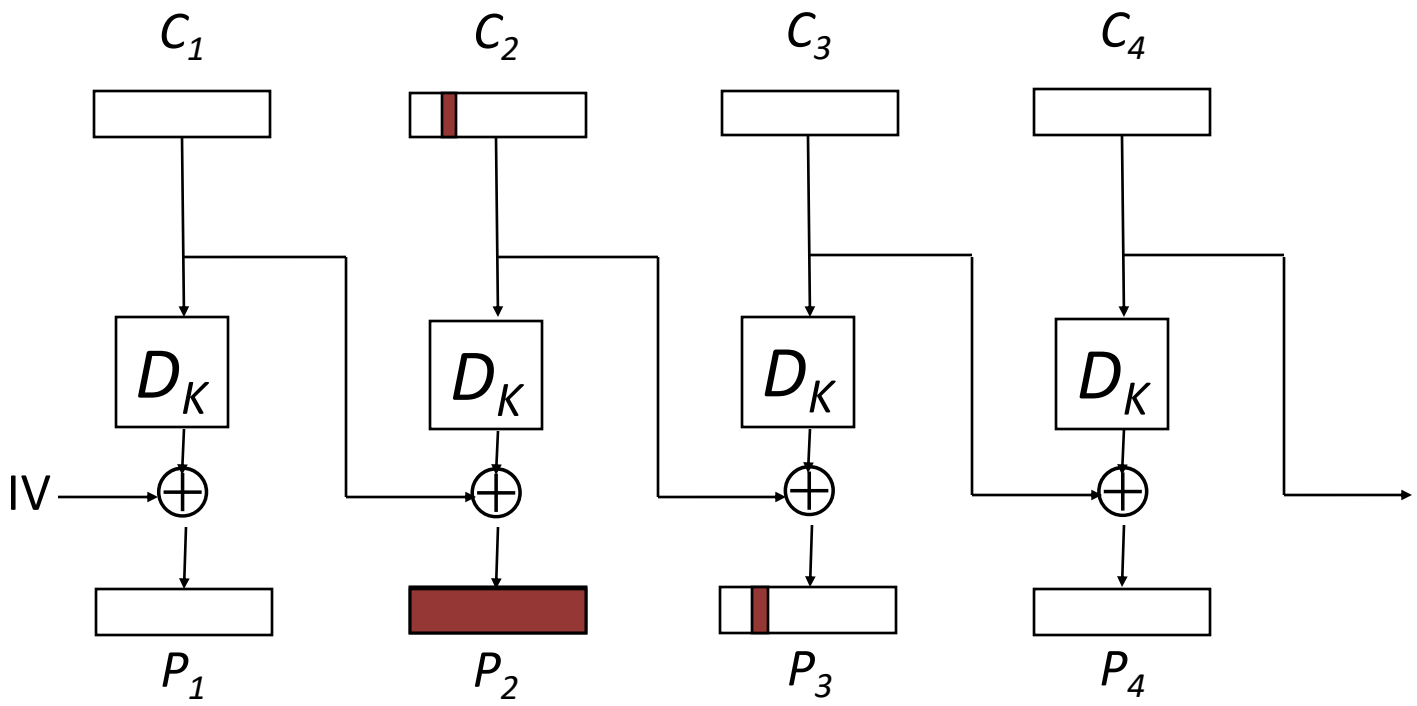
- Adversary can exploit padding errors to do a full plaintext recovery attack!
- Suppose the adversary has access to a *padding oracle*:



- Adversary can submit arbitrary ciphertexts *C* and learn whether the underlying plaintexts have valid or invalid padding
- This models information the adversary may learn by practical application behaviour

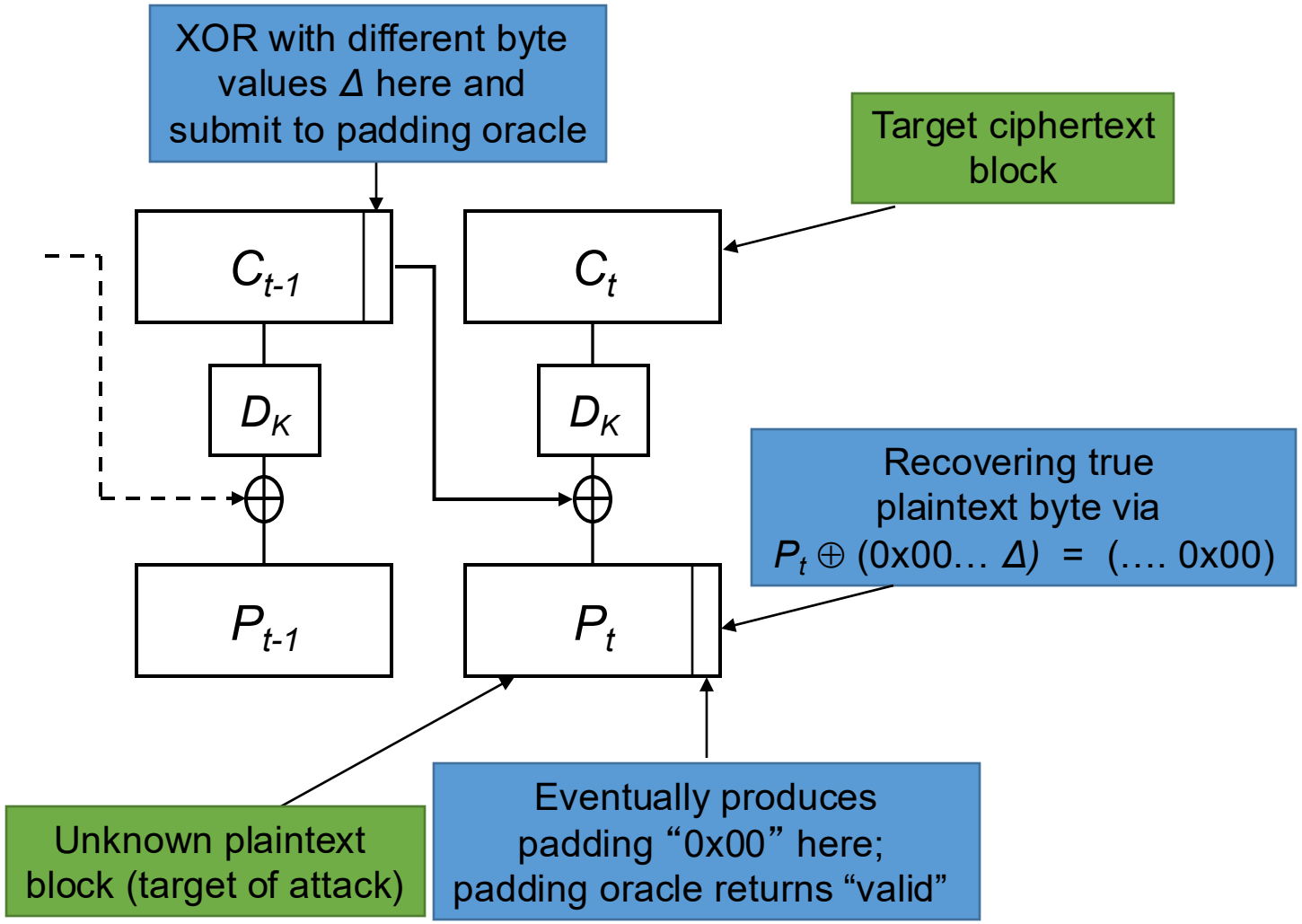
Recall: error propagation in CBC

- Suppose an error arises in ciphertext block C_2

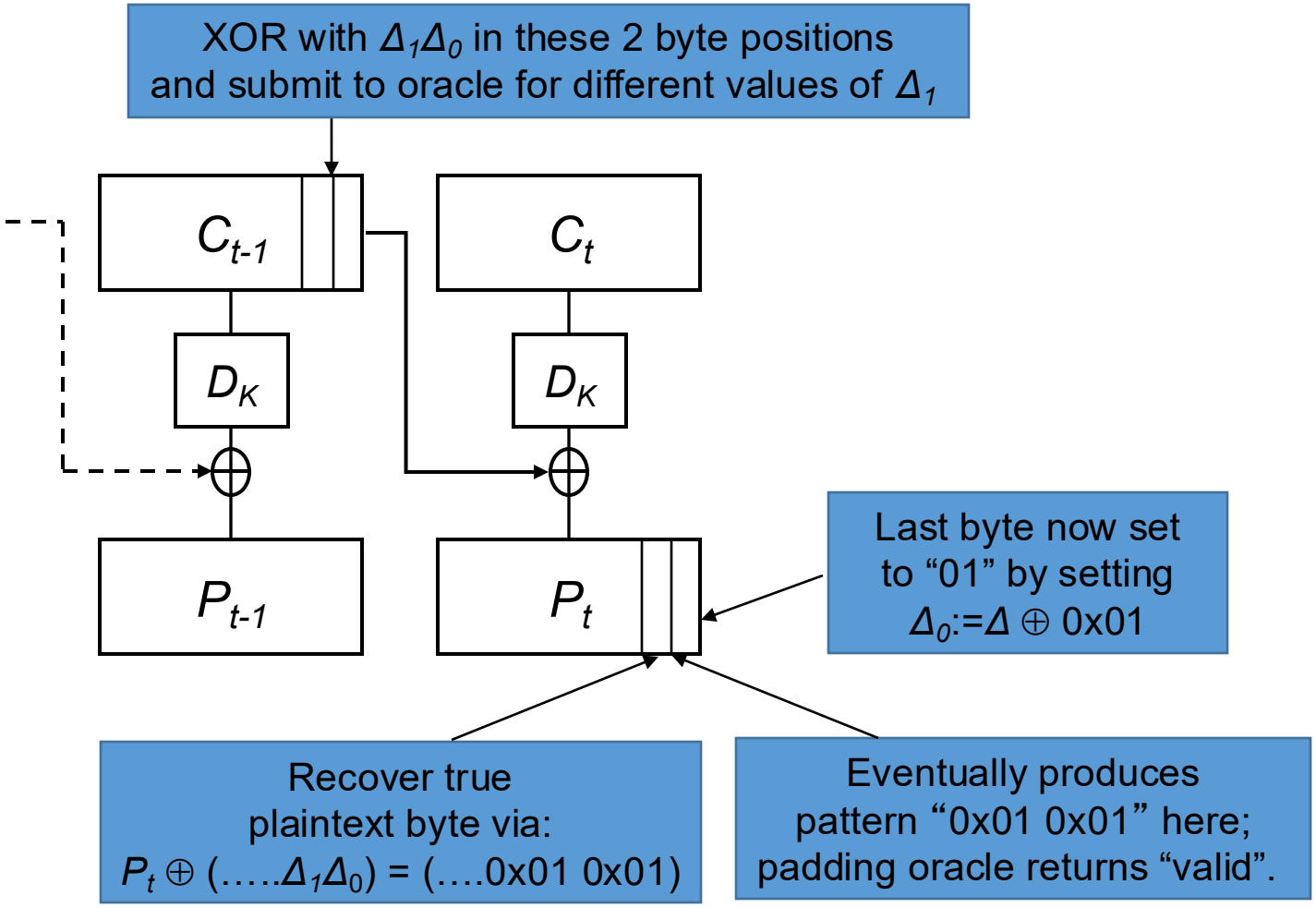


- Then the error propagates to P_3, and P_2 gets randomized
- if we replace C_2 by $C_2 \oplus \Delta$ then the effect is to replace P_3 by $P_3 \oplus \Delta$

Padding oracle attack for simplified TLS padding
Learning the last byte



Padding oracle attack for simplified TLS padding
Learning the next-to-last byte



Padding oracle attack for simplified TLS padding

- An average of 128 calls to the padding oracle are needed to extract the last byte of each plaintext block
- Can extend to the entire block, with an average of 128 trials per byte
  - Target bytes from right to left in block
  - Gradually increase length of valid padding pattern
- Can extend to an entire ciphertext with many blocks
  - Attacker can place *any* target block C_t as last block of ciphertext

Further remarks

- Original concept of padding oracle attack is due to Vaudenay (2002)
  - A special kind of chosen ciphertext attack
- Padding oracles are endemic in cryptographic implementations and continue to be discovered to this day
    - Practical examples: XML encryption, ASP.NET, SSL/TLS, DTLS
    - The SSL/TLS case is particularly interesting: attack was discovered in 2002/2003, patched, but then rebooted by the “Lucky 13” attack in 2013
    - More details in later lecture
- Main take-away: padding is a delicate business, and CBC mode with padding is generally vulnerable
- Additional protection for CBC mode is needed – integrity via MACs
  - We will introduce MACs in later lecture

Homework

- Read Section 4.6 of *Everyday Cryptography*
- Read Chapter 4 of *Serious Cryptography*
- Check the "*Further Reading & Research*" section in Moodle for more details about the attacks

- In Lab 2 you will carry out a padding oracle attack!